Análisis de Complejidad Temporal y Rendimiento de Algoritmos de Ordenamiento en Sistemas Embebidos

BADILLO GARCIA JOSE MIGUEL,LOPEZ HERNANDEZ JESUS FRANCISCO,CONTRERAS JUAREZ ARMANDO
FACULTAD DE ESTUDIOS SUPERIORES CUAUTITLAN
UNIVERSIDAD AUTONOMA DE MEXICO
09/03/2026

***Resumen*—El presente trabajo documenta la implementación, análisis teórico y medición de dos algoritmos de ordenamiento aplicados a un arreglo de 100 números enteros generados aleatoriamente. El objetivo principal es contrastar la complejidad computacional teórica dictada por la notación O (Big-O) frente a los tiempos de ejecución y ciclos de reloj reales medidos a través del depurador de Microchip Studio. Los resultados permiten comprender el impacto directo de la elección algorítmica en la eficiencia de procesamiento dentro de arquitecturas con recursos limitados.**

I. Introducción

Este trabajo presenta un programa en ensamblador para el microcontrolador ATmega328P que genera 100 números pseudoaleatorios en la memoria `table_of_unsorted_numbers`. Posteriormente, estos datos son procesados por dos algoritmos de ordenamiento distintos, cuyos resultados se almacenan de forma independiente en `table_of_sorted_numbers_alg1` y `table_of_sorted_numbers_alg2`.

El objetivo principal es comprender el funcionamiento de los algoritmos de ordenamiento a bajo nivel, así como practicar el manejo de memoria, registros y estructuras de control en lenguaje ensamblador dentro de una arquitectura AVR. Esto permite analizar cómo se realizan estas operaciones directamente en el hardware y evaluar la eficiencia computacional de cada algoritmo implementado.

II. Teoría e Importancia de los Algoritmos de Ordenamiento

Los algoritmos de ordenamiento son secuencias finitas de instrucciones diseñadas para reorganizar un conjunto de datos bajo un criterio específico. Su importancia en la ingeniería radica en:

- **Optimización de Búsqueda:** Algoritmos eficientes como la búsqueda binaria requieren que los datos estén previamente ordenados, reduciendo el tiempo de búsqueda de $O(n)$ a $O(\log n)$.
- **Eficiencia en Hardware:** En microcontroladores, un algoritmo mal optimizado consume más ciclos de instrucción, lo que aumenta el consumo de energía y bloquea la CPU.
- **Gestión de Memoria:** Algunos algoritmos ordenan ïnplace"(sin usar memoria extra), crítico en arquitecturas de RAM escasa.

III. Notación Big-O en Ciencias de la Computación

La notación Big-O (O) es una herramienta matemática utilizada para describir el límite superior del tiempo de ejecución o el espacio en memoria que requiere un algoritmo en función del tamaño de la entrada (n). Permite predecir el comportamiento del programa a medida que los datos crecen. En terminos simples la notación Big O define el peor de los casos de un algoritmo. Si tienes una lista de n elementos, el Big O te dice cuántas operaciones, en proporción a n, tendrá que hacer el procesador.

Tabla I
Tabla Comparativa de Complejidad Algorítmica

Notación	Nombre	Eficiencia	Descripción Técnica
$O(1)$	Constante	Excelente	El tiempo de ejecución no depende del tamaño de n.
$O(\log n)$	Logarítmica	Muy Buena	El espacio de búsqueda se reduce a la mitad en cada iteración.
$O(n)$	Lineal	Buena	El tiempo crece en proporción directa a la cantidad de elementos.
$O(n \log n)$	Lineal-Logarítmica	Aceptable	Combinación de partición de datos y procesamiento lineal.
$O(n^2)$	Cuadrática	Pobre	El tiempo aumenta proporcional al cuadrado de la entrada.
$O(2^n)$	Exponencial	Deficiente	El crecimiento se duplica con cada incremento unitario de n.
$O(n!)$	Factorial	Crítica	El crecimiento es masivo; inviable para valores de n superiores a 15-20.

IV. Pseudocódigo de los Programas

A continuación, se define la lógica para la generación de datos y dos algoritmos de complejidad cuadrática $O(n^2)$ implementados para las pruebas de depuración.

Algoritmo 1 Ordenamiento de Burbuja (Código en Ensamblador AVR)

Datos: Arreglo desordenado en `table_of_sorted_numbers_alg1`
Resultado: Arreglo ordenado de forma ascendente

```
inicio
BUBBLE_SORT:
    ldi r19, 100    ; Límite exterior (n elementos)
OUTER_B:
 ldi ZL, low(table_of_sorted_numbers_alg1)
 ldi ZH, high(table_of_sorted_numbers_alg1)
    mov r20, r19                ; Límite interior
    dec r20
    breq DONE_B                 ; Finalizar si n = 1
INNER_B:
    ld r0, Z            ; Carga elemento actual (i)
    ldd r1, Z+1 ; Carga elemento siguiente (i + 1)
    cp r1, r0           ; Compara (i + 1) con (i)
    brsh NO_SWAP_B      ; Si (i + 1) ≥ (i), no hay intercambio
    ; — Intercambio de valores —
    st Z, r1 ; Almacena el menor en la posición (i)
    std Z+1, r0 ; Almacena el mayor en la posición (i + 1)
NO_SWAP_B:
    adiw ZL, 1          ; Incrementa puntero Z
    dec r20
    brne INNER_B        ; Repetir bucle interior
    dec r19
    brne OUTER_B        ; Repetir bucle exterior
DONE_B:
    ret
fin
```

Análisis de la Implementación 1er algoritmo:
Este algoritmo recorre la tabla comparando elementos adyacentes e intercambiándolos si están en el orden incorrecto. Utiliza la instrucción LDD (Load with Displacement) para optimizar el acceso a la memoria sin desplazar el puntero base en cada comparación.

Análisis de la Implementación 2do algoritmo: Este algoritmo busca el valor mínimo en la sublista no ordenada y lo intercambia con el primer elemento de dicha sublista. Minimiza la cantidad de escrituras en la SRAM, realizando un máximo de un intercambio por cada iteración del bucle externo.

Algoritmo 2 Ordenamiento por Selección (Código en Ensamblador AVR)

Datos: Arreglo desordenado en `table_of_sorted_numbers_alg2`
Resultado: Arreglo ordenado de forma ascendente

```
inicio
SELECTION_SORT:
    ldi XL, low(table_of_sorted_numbers_alg2)
    ldi XH, high(table_of_sorted_numbers_alg2)
    ldi r19, 99 ; n-1 pasadas para el bucle externo
OUTER_S:
    mov ZL, XL   ; Puntero de búsqueda inicia en la posición de X
    mov ZH, XH
    ld r21, Z+     ; Carga el primer valor como el mínimo inicial
    mov r24, XL ; Almacena la dirección del mínimo (R25:R24)
    mov r25, XH
    mov r20, r19       ; Contador para el bucle de búsqueda interna
INNER_S:
    ld r18, Z       ; Carga el siguiente valor para comparación
    cp r18, r21         ; ¿Dato actual < Mínimo encontrado?
    brsh NEXT_S         ; Si no es menor, continuar búsqueda
    mov r21, r18  ; Actualizar nuevo valor mínimo
    mov r24, ZL      ; Guardar dirección del nuevo mínimo detectado
    mov r25, ZH
NEXT_S:
    adiw ZL, 1  ; Avanzar a la siguiente posición de memoria
    dec r20
    brne INNER_S
    ; — Intercambio final (Swap) entre X y el valor mínimo hallado —
    ld r18, X       ; Valor en la frontera ordenada
    st X, r21    ; Colocar el mínimo en su posición definitiva
    mov ZL, r24   ; Recuperar dirección del mínimo original
    mov ZH, r25
    st Z, r18    ; Colocar el valor desplazado en la posición del mínimo
    adiw XL, 1   ; Desplazar la frontera de la tabla ordenada
    dec r19
    brne OUTER_S ; Repetir proceso hasta completar la tabla
    ret
fin
```

IV-A. *Tablas y Gráficas*

Tabla II
TABLA DE PREDICCIONES BASADA EN COMPLEJIDAD $O(n^2)$

Cantidad (n)	**Factor** (n^2)	**Est. Burbuja** (μs)	**Est. Selección** (μs)
10	$0{,}1^2 = 0{,}01$	~ 43	~ 32
50	$0{,}5^2 = 0{,}25$	$\sim 1{,}078$	~ 813
100 (Base)	$1^2 = 1{,}0$	4,314	3,255
150	$1{,}5^2 = 2{,}25$	$\sim 9{,}706$	$\sim 7{,}323$
200	$2^2 = 4{,}0$	$\sim 17{,}256$	$\sim 13{,}020$

Tabla III
RESULTADOS EXPERIMENTALES DE TIEMPO DE EJECUCIÓN

Cantidad de Números (n)	**Tiempo Burbuja** (μs)	**Tiempo Selección** (μs)
10	46.38	40.69
50	1,104.94	843.19
150	9,674.75	7,229.69
200	17,109.75	12,777.56

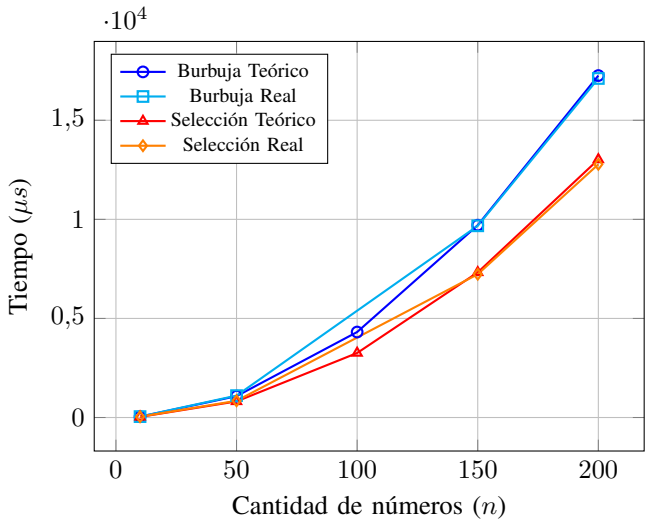

Figura 1. Comparación entre tiempos teóricos y experimentales de los algoritmos de ordenamiento.

V. ANÁLISIS DE RESULTADOS

Teniendo los datos prácticos que se muestran en las figuras 2 y 3, que son para un valor de 100 números, tenemos una relación bastante cercana con los valores dados por la fórmula y los valores teóricos. Como se muestra en la gráfica, la figura 1 nos muestra la pequeña variación en los valores.

Figura 2. Captura del depurador mostrando el tiempo de ejecucion de el algoritmo Bubble short

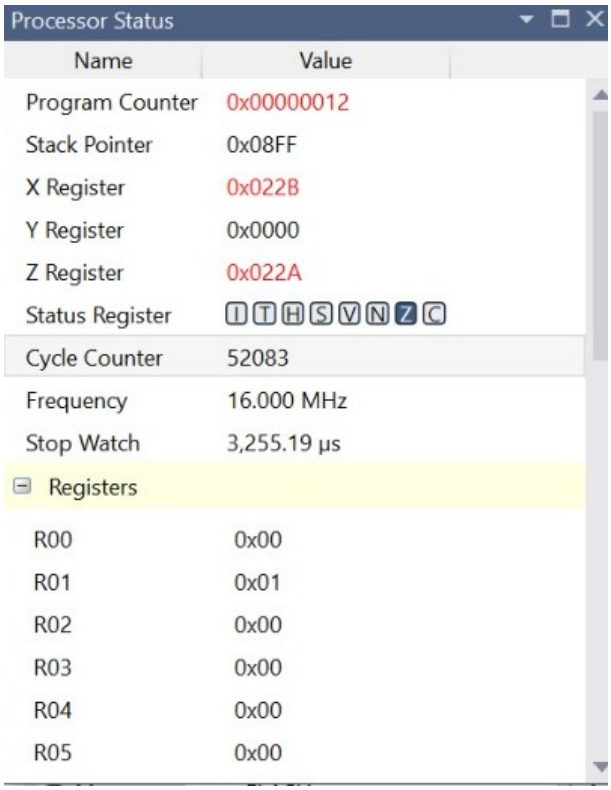

Figura 3. Captura del depurador mostrando el tiempo de ejecucion del algoritmo selection short

VI. Conclusiones

Validación de la Eficiencia Experimental:
Los resultados obtenidos en la simulación ($4{,}314\,\mu s$ para Bubble Sort y $3{,}255\,\mu s$ para Selection Sort) confirman que, aunque ambos algoritmos poseen una complejidad computacional teórica de $O(n^2)$, el impacto de la arquitectura de hardware es determinante.

La reducción del 24.5 % en el tiempo de ejecución a favor de Selection Sort se atribuye directamente a la minimización de operaciones de escritura en la memoria SRAM. Mientras que Bubble Sort realiza múltiples intercambios por cada iteración, Selection Sort garantiza un máximo de n intercambios totales, optimizando así el uso del bus de datos del microcontrolador.

VII. Referencias

Referencias

[1] Big O Notation Explained with Examples - freeCodeCamp.org. Disponible en: https://www.freecodecamp.org/news/big-o-notation-explained-with-examples/

[2] Sorting Algorithms - GeeksforGeeks. Disponible en: https://www.geeksforgeeks.org/dsa/sorting-algorithms/

[3] Microchip Technology Inc., *AVR® Instruction Set Manual*. [En línea]. Disponible en: https://www.microchip.com/webdoc/avrassembler/

[4] Algoritmos de ordenación explicados con ejemplos en JavaScript, Python, Java y C++ - freeCodeCamp.org. Disponible en: https://www.freecodecamp.org/espanol/news/algoritmos-de-ordenacion-explicados-con-ejemplos-en-javascript-python-java-y-c/